Data Science

Introduction to Deep Learning Frameworks and Tools

Deep Learning



GENERAL ASSEMBLY

Lesson roadmap

Welcome + Warm-up

A Brief History of Neural Networks

How Neural Nets Work

Knobs & Levers

Model Cards

Deep Learning Apps

Bring It Home

Learning objectives



1. Describe the evolution of Neural Networks from idea to State-of-the-Art (SOTA) modeling.
2. Define Neural Networks & how they work.
3. Build a neural network with a common Python framework (PyTorch).
4. Discuss model cards and selecting models for specific use cases.
5. Identify deep learning applications.

A Brief History of Neural Networks

First: A Brief History of Neural Networks

While Neural Networks recently gained popularity, they are not new. Research on what we now call **Deep Learning** can be traced all the way back to the 1940s. It's gone through several names:

- **Cybernetics** (1940s - 1960s).
- **Connectionism** (1980s - 1990s).
- **Deep Learning / artificial neural networks** (term first used in 2006).

 © 2025 General Assembly

So... why are we just hearing about them now?

The reason neural networks didn't gain traction until recently is because all "good" implementations require **a lot of computational power**.

Even basic neural nets require a level of computing unavailable on 1960s "supercomputers".

Small neural nets were **feasible** back then, they just didn't perform as well as more classical approaches (linear regression, logistic regression, namely).

 © 2025 General Assembly

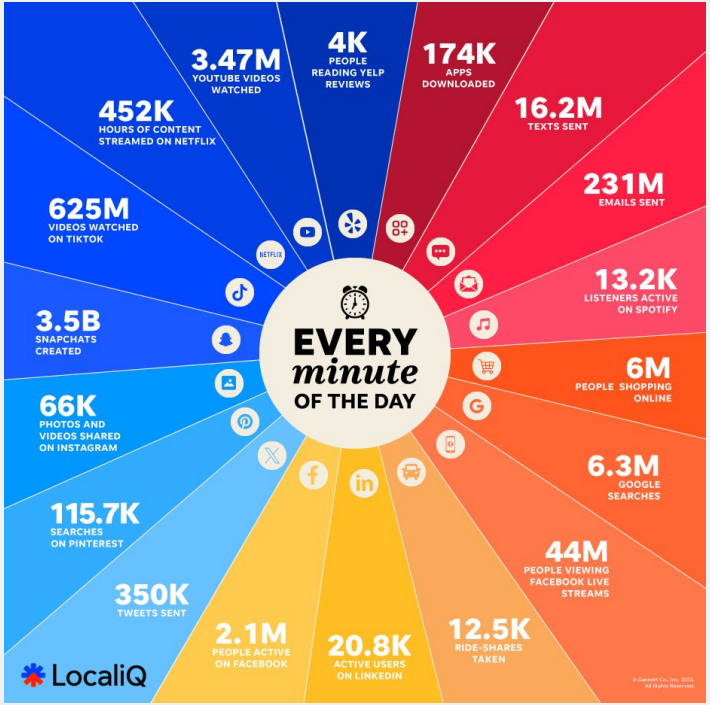
What changed?

Three things led to the modern feasibility of neural networks that we enjoy today:

1. Increased computing ability. The fact that you can train a multi-layer neural net on your personal laptop is incredible.

2. Increased algorithmic efficiency. Smarter math leads to fewer computations.

3. Increased availability of data. Neural nets need a lot of data to work properly. There is no shortage of data these days.



 © 2025 General Assembly

How Neural Nets Work

Why the name?

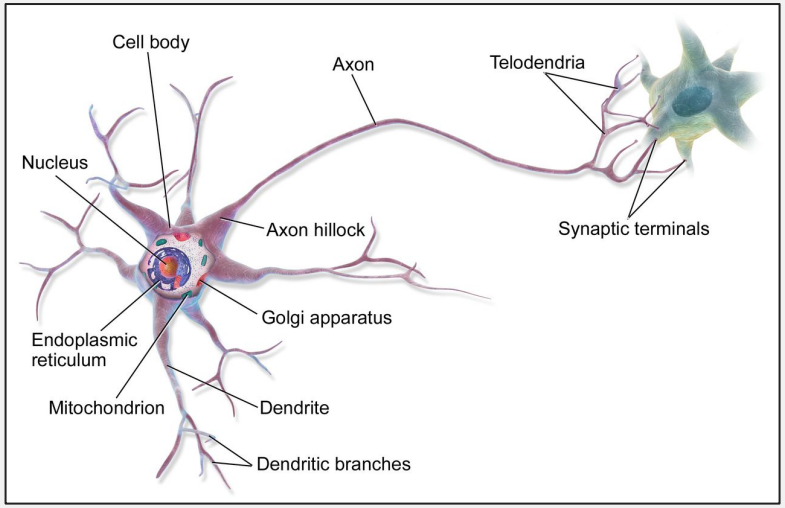
- Initially, neural nets were inspired by the way our brains' neurons work.
- This inspiration didn't last long.
- And then we found out that's not how brains work.
- The name stuck anyway.

What is a Neural Network?

Neural networks are **statistical models**.

Sometimes, researchers will incorporate loose inspirations from neural biology.

Sometimes, they lead to improvements.

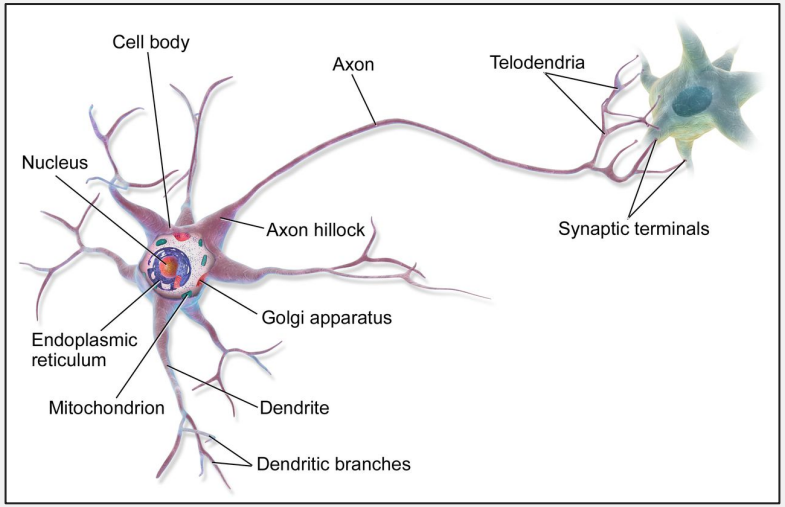


Source

 © 2025 General Assembly

“Neural Nets”

The term caught on in the press early on because it was believed neural networks mimic the way our brains’ neurons process information.



Source

 © 2025 General Assembly

Neural Networks

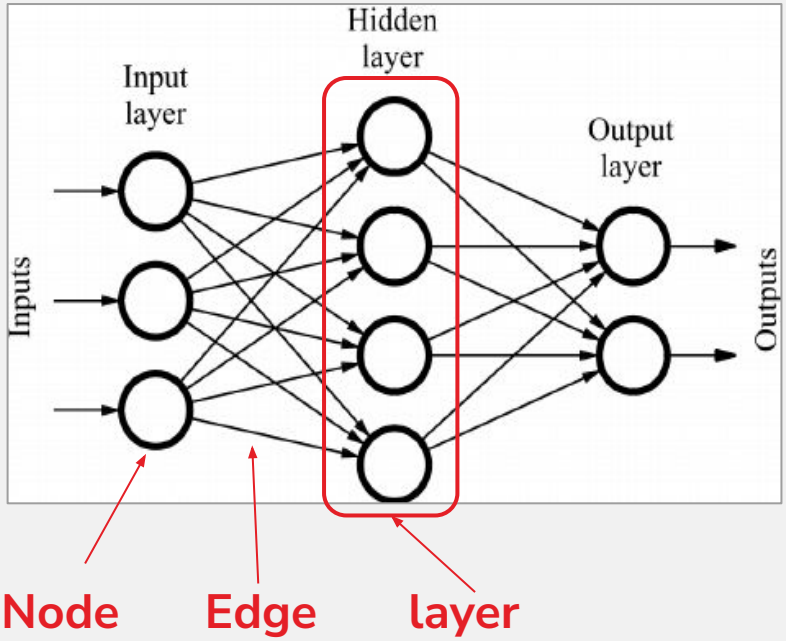
Neural Network: takes input data, passes it through layers of neurons, and produces an output.

Nodes = Neuron

Edges = Connections (Weights)

Layers = Groups of Neurons

Graph of our Neural Network

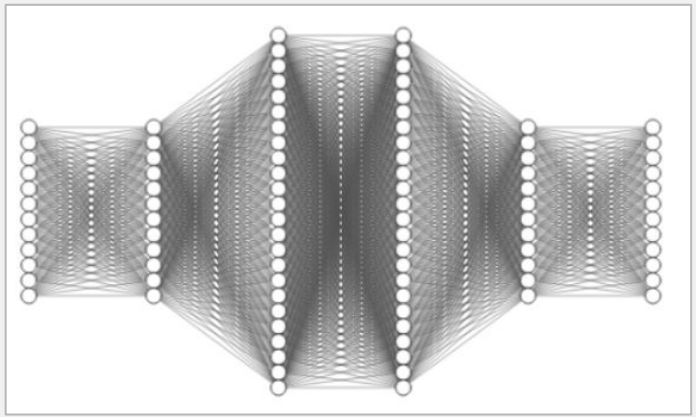


 © 2025 General Assembly

Source

“Deep Learning”

The term “Deep Learning” comes from the fact that you often have many hidden layers, making your graph very deep.





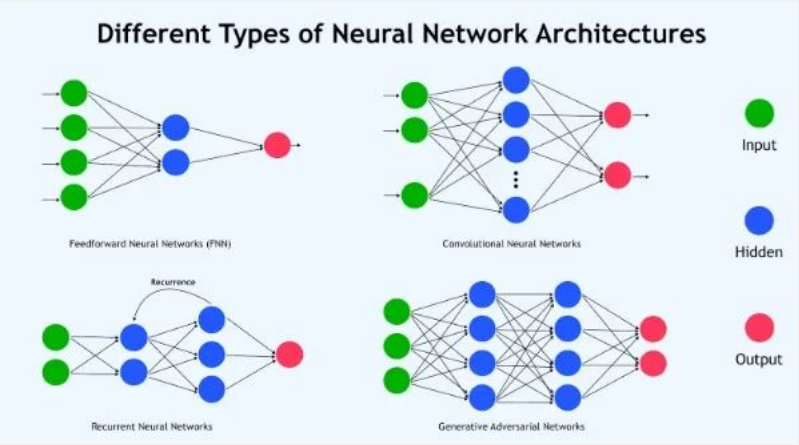
 © 2025 General Assembly

Source 1 Source 2

Uses for Neural Networks

Neural nets have found numerous uses. It's harder to find places where you can't use a neural net.

Though it can be more complex, what will be most interesting to us, is its flexibility to work with many ***kinds*** of data.

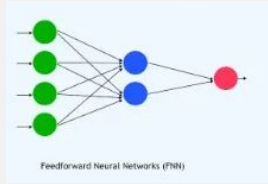


**GANs are beyond the scope of this course.*

 © 2025 General Assembly

Source: Google

Traditional Tabular Data



Working with the “usual” kind of data you see in an excel spreadsheet or csv is done with **densely connected neural networks** (referred to as DNNs, Dense Neural Networks, FFNs or called “Feed Forward Networks”), which is what we’ll focus on first.

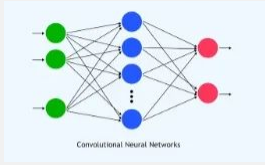
You’ll learn to implement them in **PyTorch** in a future lesson.

	age (n)	job (c)	marital (c)	education (c)	balance (n)	housing (c)
0	30	unemployed	married	primary	1787	no
1	33	services	married	secondary	4789	yes
2	35	management	single	tertiary	1350	yes
3	30	management	married	tertiary	1476	yes
4	59	blue-collar	married	secondary	0	yes
5	35	management	single	tertiary	747	no

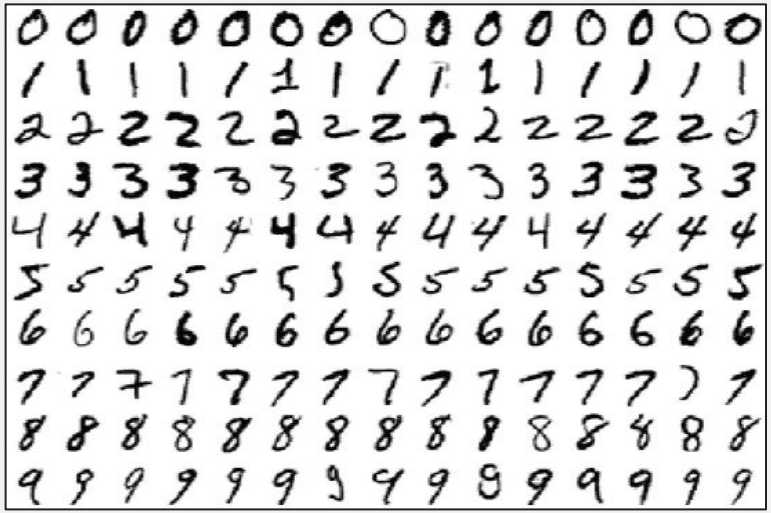
Source

 © 2025 General Assembly

Image Data



Convolutional Neural Networks (CNNs) are a variety of neural networks especially equipped to work with image data.

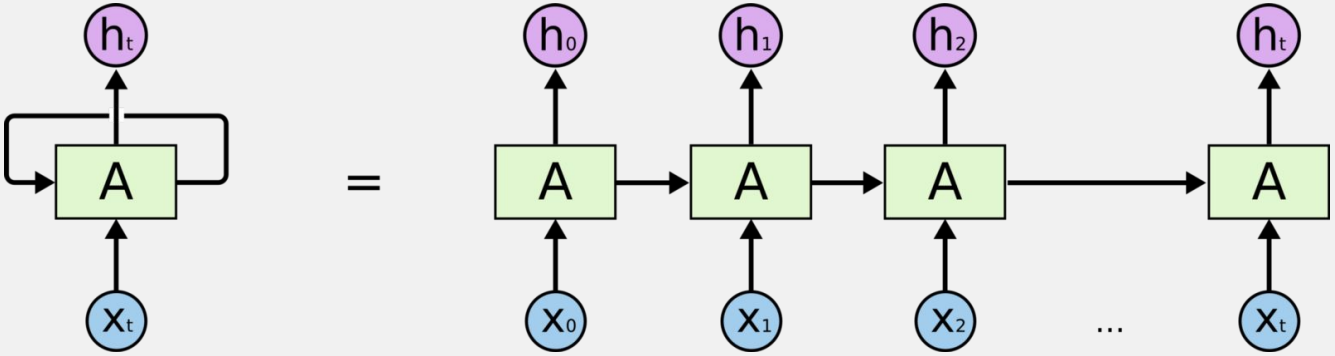


 © 2025 General Assembly

Source

Sequence Data

Recurrent Neural Networks (RNNs) are a variety of neural networks that specialize in working with sequence data, such as time series or natural language data.



 © 2025 General Assembly

Source

Everyday Applications of Neural Networks

- **Image recognition** (e.g., classifying photos, facial recognition on phones)
- **Voice assistants** (e.g., Siri, Alexa, Google Assistant: speech-to-text and intent detection)
- **Email spam detection** (filtering unwanted messages)
- **Autocorrect and predictive text** (next-word prediction in keyboards)
- **Recommendation systems** (e.g., Netflix, YouTube, Spotify suggestions)

 © 2025 General Assembly

Advanced and Industrial Applications

- **Medical diagnosis** (e.g., detecting tumors in X-rays or MRIs)
- **Fraud detection** (e.g., flagging suspicious credit card activity)
- **Self-driving cars** (e.g., detecting lanes, pedestrians, road signs)
- **Language translation** (e.g., Google Translate using sequence models)
- **Robotics** (e.g., smart navigation and task execution in real-world environments)

 © 2025 General Assembly

Creative and Cutting-edge Uses

- **Image generation** (e.g., DALL·E, Stable Diffusion creating art from text)
- **Music composition** (AI generating original melodies or full songs)
- **Chatbots and virtual agents** (e.g., customer service bots using LLMs)
- **Stock market prediction** (e.g., modeling time series with RNNs/LSTMs)
- **Personalization engines** (e.g., tailoring shopping or news experiences)

 © 2025 General Assembly

Recap: Logistic Regression

Let's suppose I have some *x*-variables, and I want to use them to predict a binary 0/1 *y*-variable using logistic regression:

$$\text{logit}\, p = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3$$

Let's rewrite:

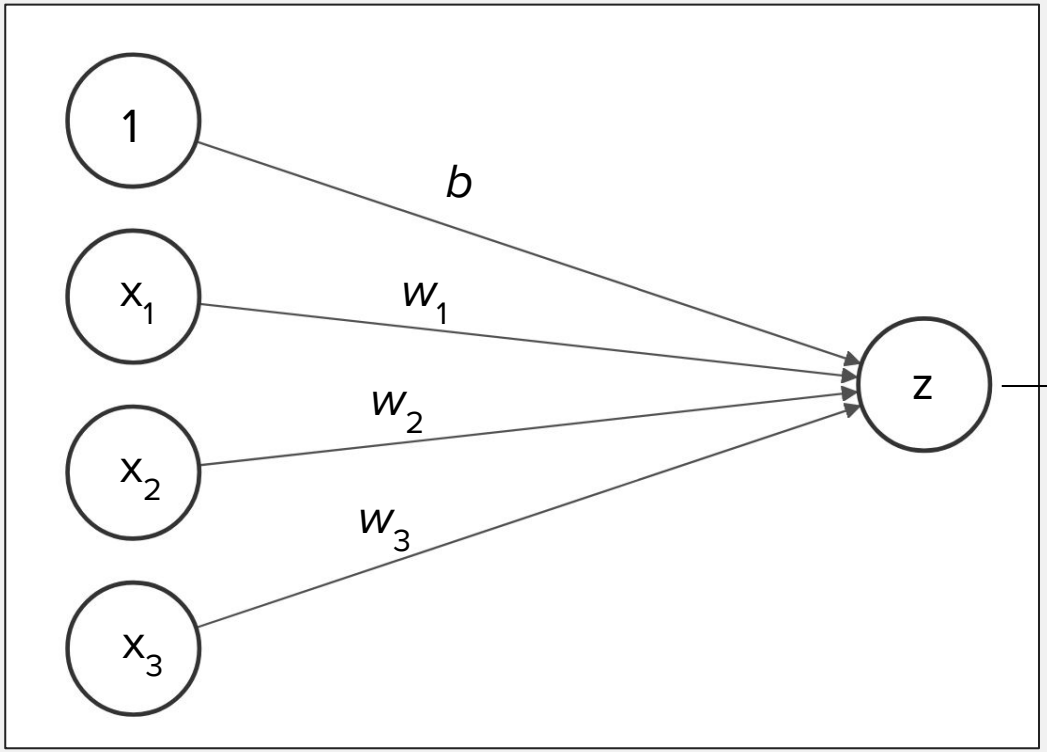
$$p = \sigma\left(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3\right)$$

Let's refer to the βs as "weights" with a "bias" term instead:

$$p = \sigma\left(b + w_1 x_1 + w_2 x_2 + w_3 x_3\right)$$

 © 2025 General Assembly

Let's draw this as a graph!



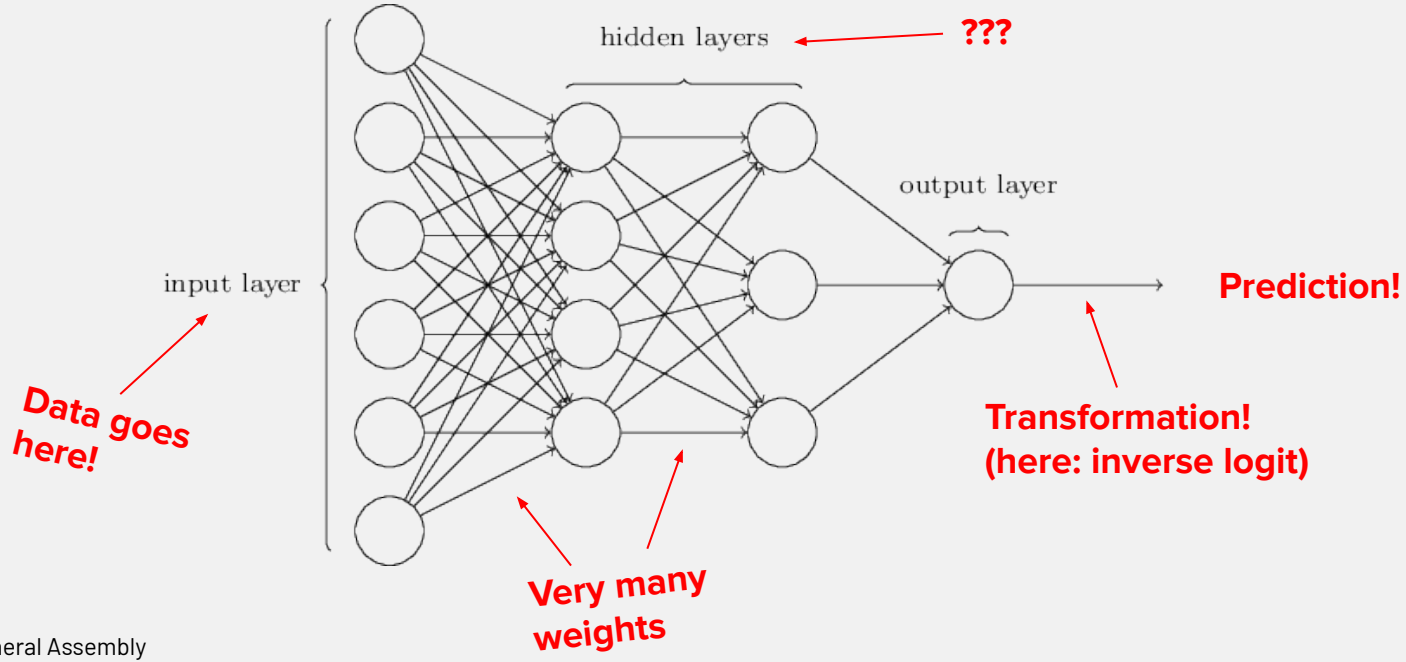
$\sigma(z)$ = prediction

$z = b + x_1 w_1 + x_2 w_2 + x_3 w_3$

 © 2025 General Assembly

What if we kept stacking logistic regressions?

Output of one gets transformed and then passed as input to another:

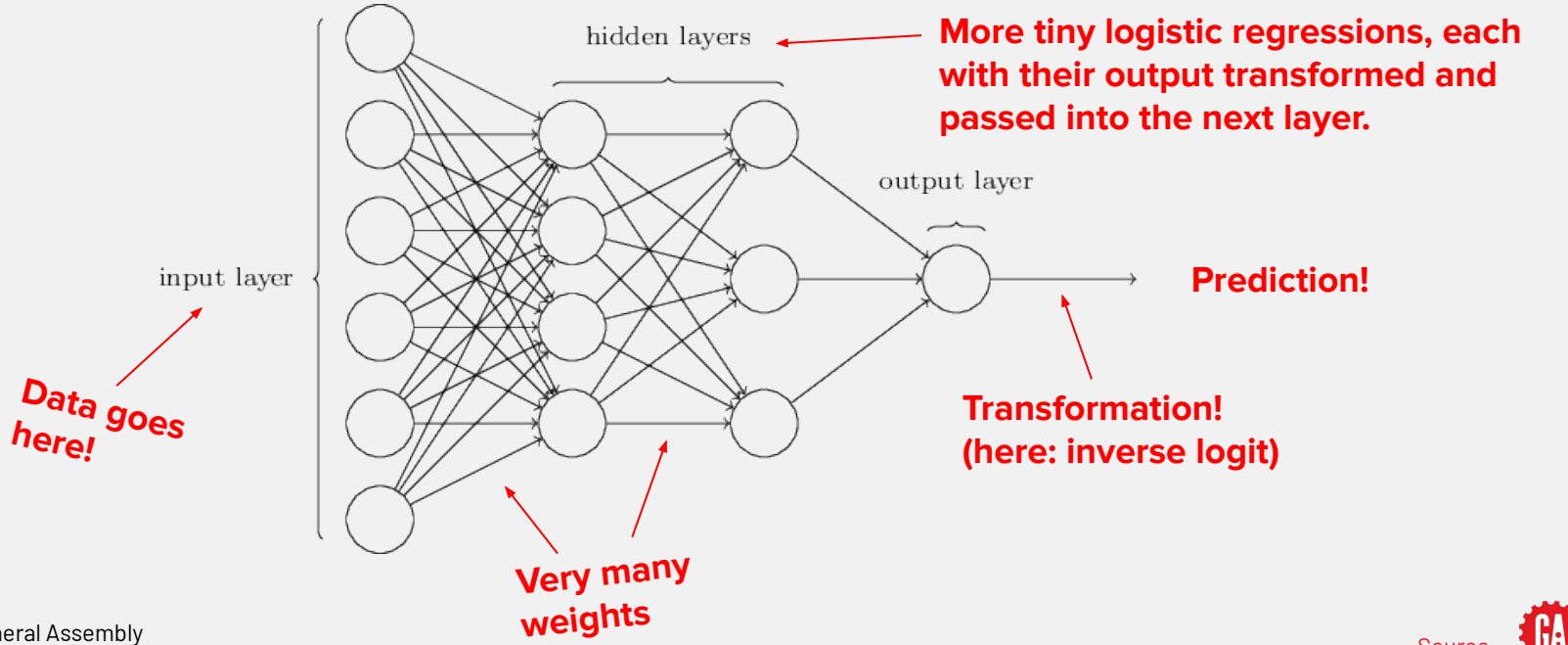


Source

 © 2025 General Assembly

What if we kept stacking logistic regressions?

Output of one gets transformed and then passed as input to another:



 © 2025 General Assembly

Source

Why did we do this?

$$p = \sigma\left(b + w_1 x_1 + w_2 x_2 + w_3 x_3\right)$$

Curvy part

Linear part

The linear parts allow our data to impact our result.

The transformation (**curvy**) part allows us to learn more complex, non-linear relationships between x and y.

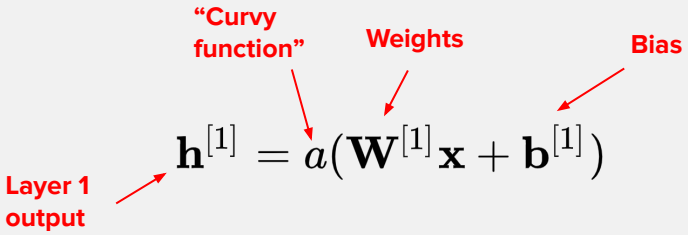
Multiple layers allow us to learn **even more wiggly and complex relationships within our data.**

Let's play!

 © 2025 General Assembly

Connecting Hidden Layers

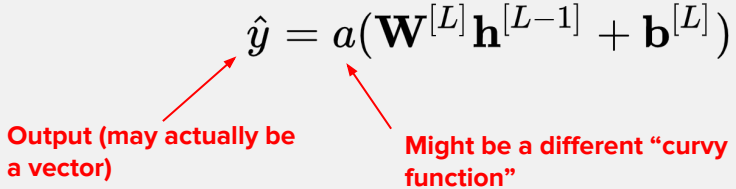
We start with:



This is then fed in as the *x*-variable to layer *two*:

$$\mathbf{h}^{[2]} = a(\mathbf{W}^{[2]}\mathbf{h}^{[1]} + \mathbf{b}^{[2]})$$

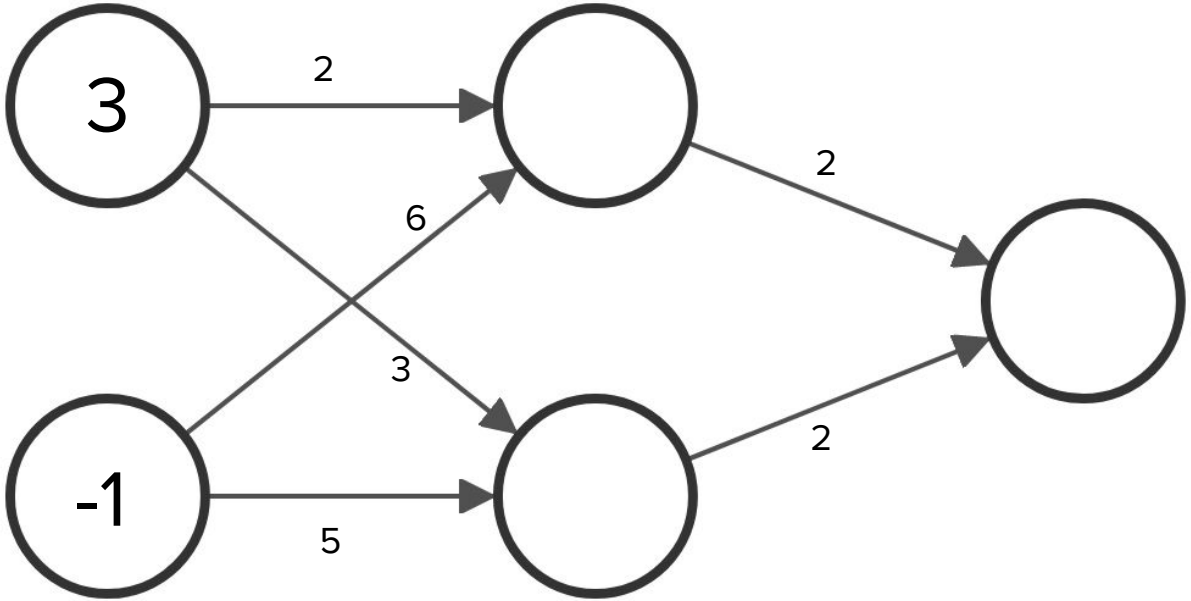
And so on until the last hidden layer (L), which give us our output:



 © 2025 General Assembly

BRAINSTORMING: What is the output?

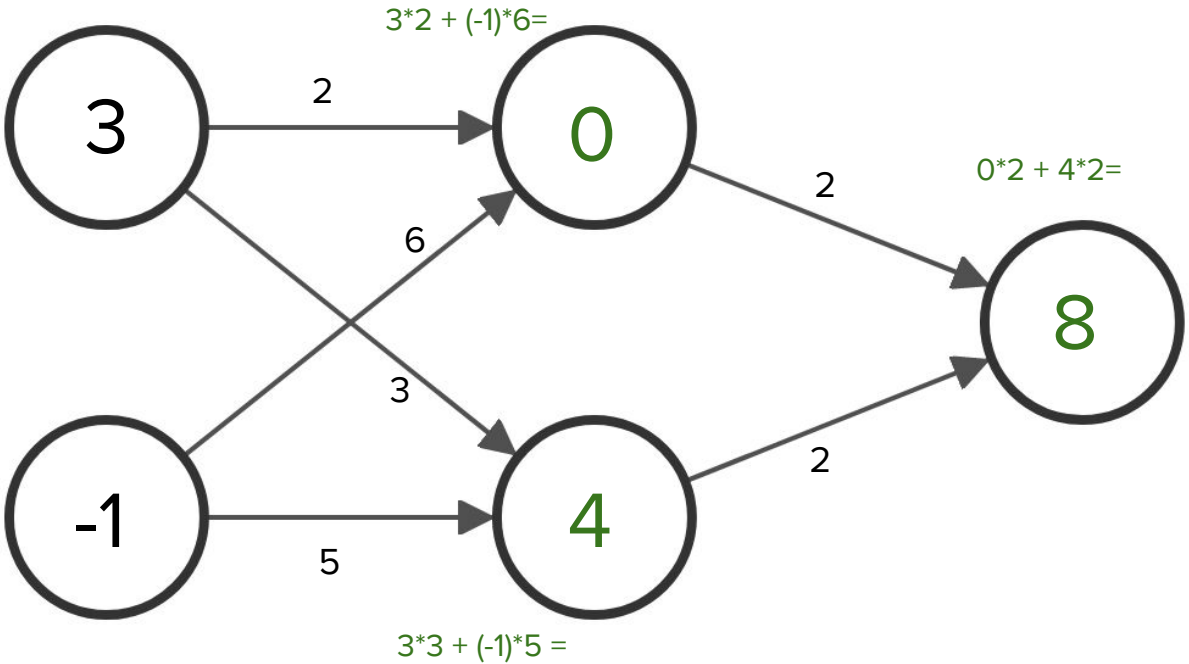
What is my output here?
(Assume no transformations or biases)



 © 2025 General Assembly

BRAINSTORMING: What is the output?

What is my output here?
(Assume no transformations or biases)



 © 2025 General Assembly

Knobs & Levers

Knobs and Levers

When constructing a neural net, you have many choices to make, such as:

- Number of hidden layers.
- Number of nodes in each hidden layer.
- Choice of activation functions.
- Loss function.
- Optimizer.
- Minibatch size.

We'll discuss what each of these mean and how to choose them.

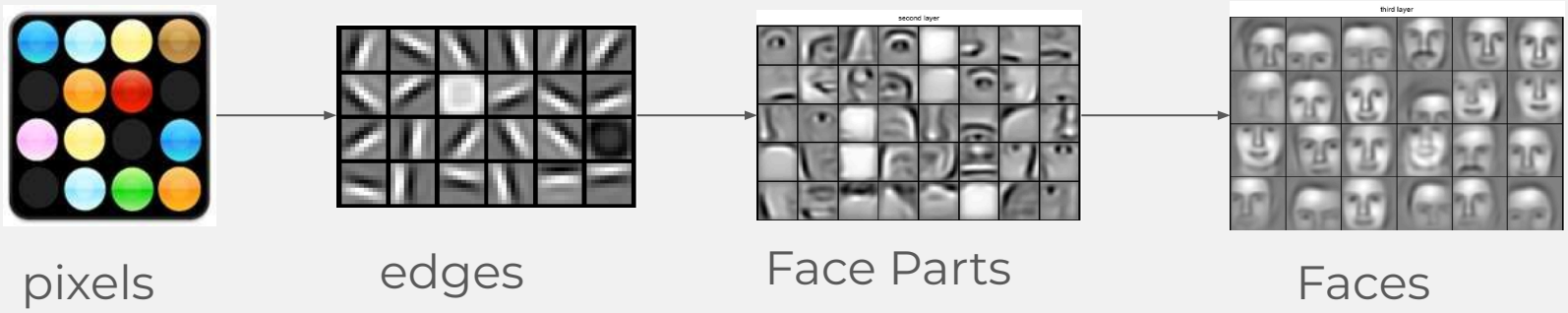
 © 2025 General Assembly

Hidden Layer Wisdom

More hidden layers = More overall complexity of relationship learned

More hidden nodes = More relationships learned (that will eventually be aggregated into prediction)

Experimenting with hidden layers & nodes will help you find the optimal model



 © 2025 General Assembly

Source

Number of Hidden Nodes and Layers

These control the complexity of your model. Too much of either will cause you to overfit. You'll often need to **regularize**.

Unfortunately, there is no hard rule on how you should construct these. They will involve a painful amount of trial and error, as your situation will always be unique.

 © 2025 General Assembly

Activation Functions

An **activation function** is a transformation function applied to the output of a layer. That is, they're the **"curvy part"**.

In our special case of a logistic regression, the activation function was the inverse logit. In deep learning literature, this actually called the **sigmoid activation** (hence why we denoted it **σ**).

There are some activation function flashcards on the next few slides.

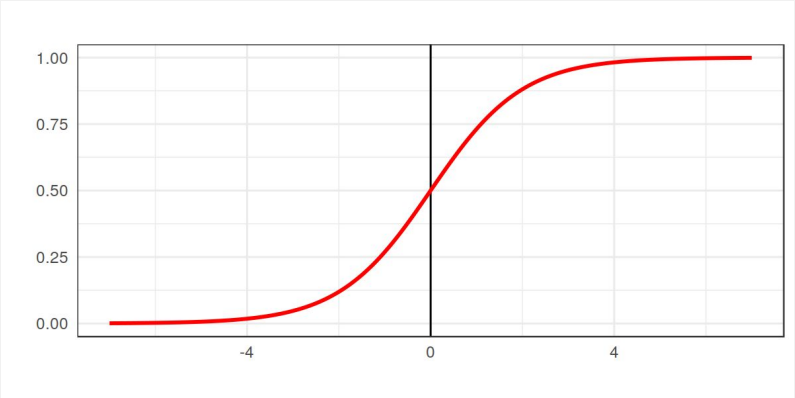
 © 2025 General Assembly

Sigmoid

Since the sigmoid function ranges from 0 to 1, it is often used as the output activation for binary classification models.

It used to be used as hidden layer activations, too, but has since been abandoned in favor of ReLU.

$$\sigma(z) = \frac{1}{1+e^{-z}}$$



 © 2025 General Assembly

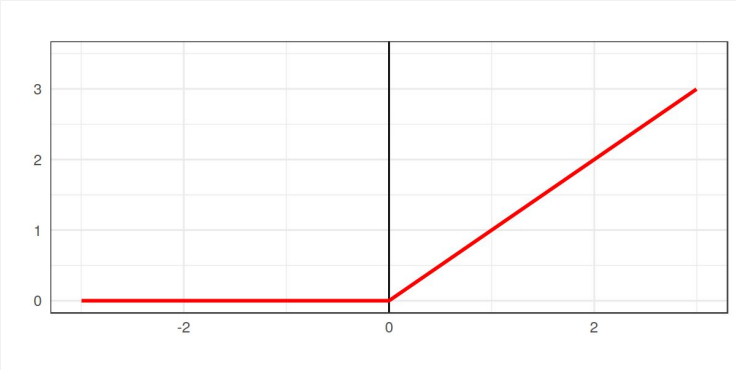
ReLU

The rectified linear unit (**ReLU**) is the most common activation for hidden layers.

It was inspired by neurobiology, since real neurons don't activate until some stimulus threshold is met.

There are variants of the **ReLU** (see: Leaky ReLU).

$$a(z) = \max\{0, z\}$$

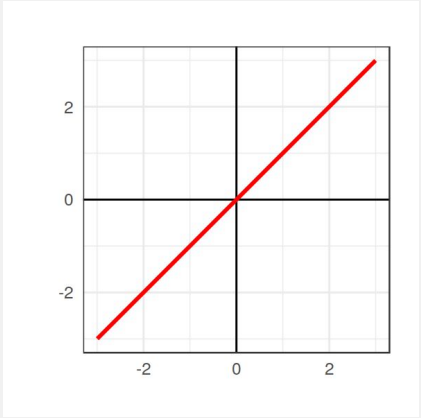


 © 2025 General Assembly

Identity/Linear

This activation does nothing. It is often used as the output for a regression problem, since it maps to the whole real line.

$$a(z) = z$$



 © 2025 General Assembly

Never use the identity activation as a hidden layer activation!

Hyperbolic Tangent

The **tanh** function is pronounced “tanch” and varies between -1 and 1.

It is sometimes used as a hidden layer activation, but usually doesn’t perform as well as ReLU.

It is sometimes used as output for binary classification, but doesn’t differ very much from sigmoid.

$$a(z) = \tanh(z)$$



 © 2025 General Assembly

Softmax

The **softmax** is an extension of the sigmoid to multiclass classification. It is used as the output activation for multiclass classification.

It is vector valued, and varies between 0 and 1 for each element, and sums to 1. **This means the softmax is like a vector of probabilities!**

$$a(z_j) = \frac{e^{z_j}}{\sum e^{z_i}}$$

 © 2025 General Assembly

Logic Behind Hidden Layer Activations

Activations in the hidden layer provide a transformation that allow the neural net to learn more complex relationships as calculations propagate through the network.

There is little need to use different activations across different hidden layers.

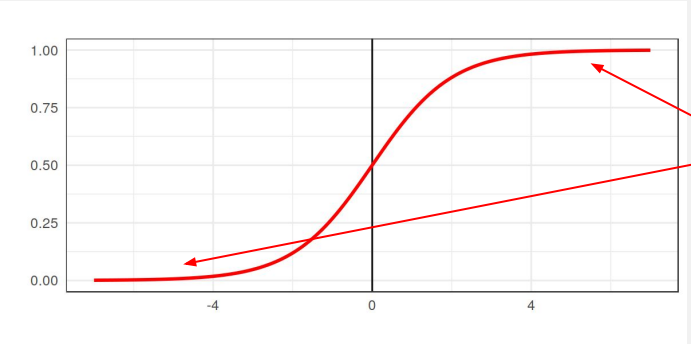


Source

 © 2025 General Assembly

Logic Behind Hidden Layer Activations

Sigmoid used to be the hidden activation of choice, but it has problems. Our optimization will be gradient-based, and the gradients are approximately zero for extreme values. This yields slow learning.



Once **ReLU** was discovered, the sigmoid was abandoned for hidden layers, and is now actively discouraged.

 © 2025 General Assembly

Output Layer Activations

The output layer activations are particularly important because those decide what form our predictions take.

Your output activation should map onto the space of values occupied by your target variable.

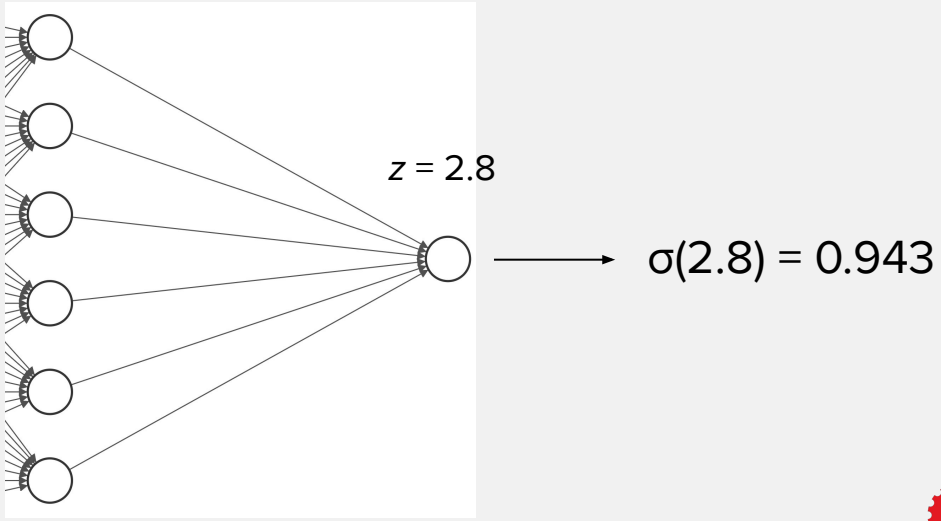
What that means is...

 © 2025 General Assembly

Output Activations: Binary Classification

If your output variable is a binary class, your best bet is the **sigmoid activation**, since it maps directly onto 0 and 1.

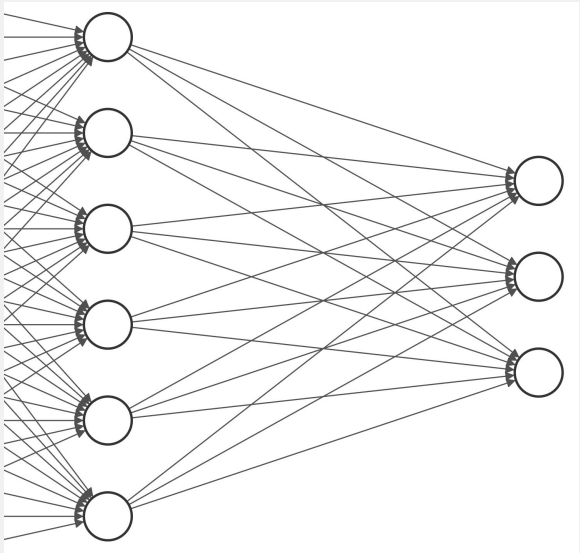
Alternatively, you could use the **tanh activation**, which maps between -1 and 1, and simply scale it afterwards.



 © 2025 General Assembly

Output Activations: Multiclass Classification

For multiclass classification, the **softmax activation** generalizes the sigmoid to many outputs. **Notice three nodes in the output layer!**



$$\begin{bmatrix} 2.8 \\ 1.5 \\ 3.7 \end{bmatrix} \longrightarrow \text{softmax}\left(\begin{bmatrix} 2.8 \\ 1.5 \\ 3.7 \end{bmatrix}\right) \longrightarrow \begin{bmatrix} 0.27 \\ 0.07 \\ 0.66 \end{bmatrix}$$

 © 2025 General Assembly

Output Activations: Regression

When carrying out regression, there is no need to transform output, since you are predicting a quantity.

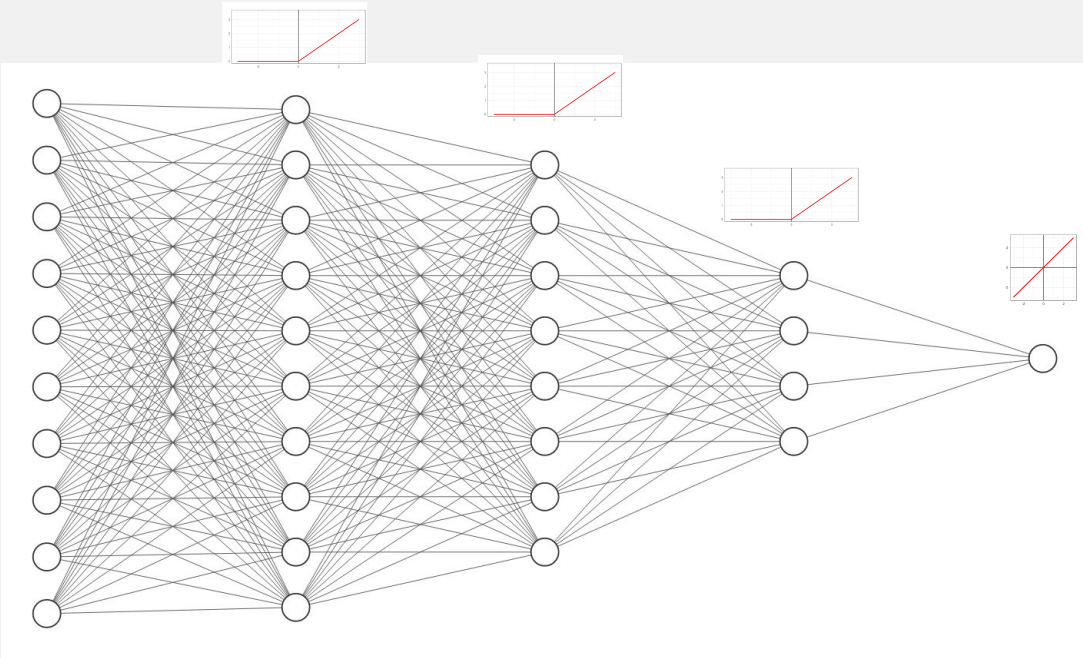
You may either specify the **linear/identity activation**, or (in most software), you can simply decline to specify an activation.



Source

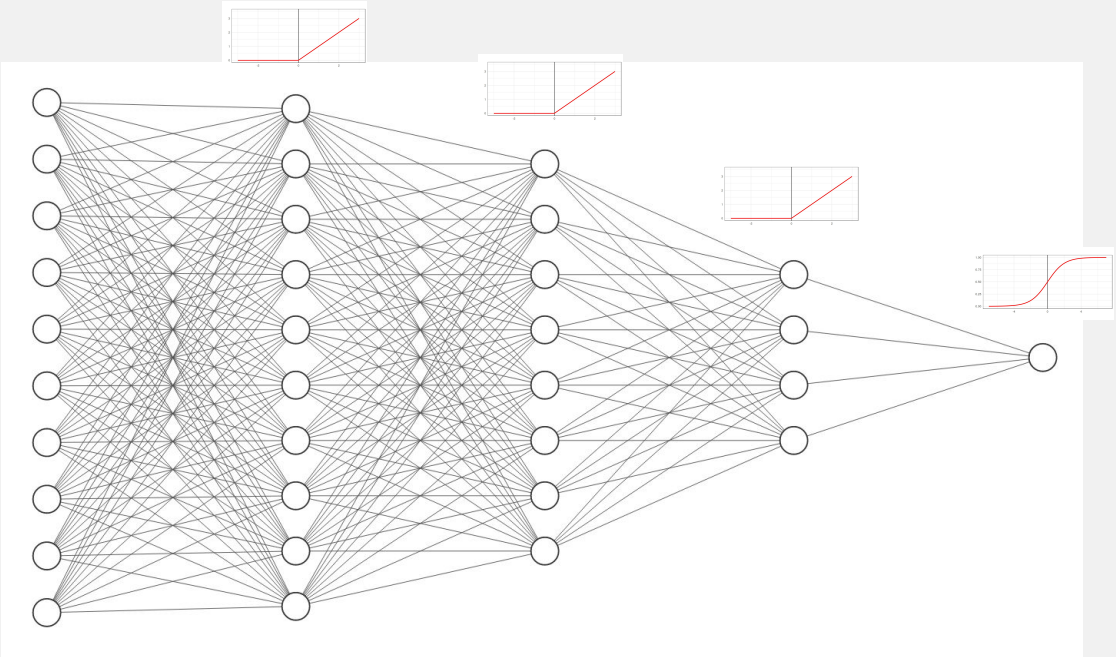
 © 2025 General Assembly

Example: Regression with NNs



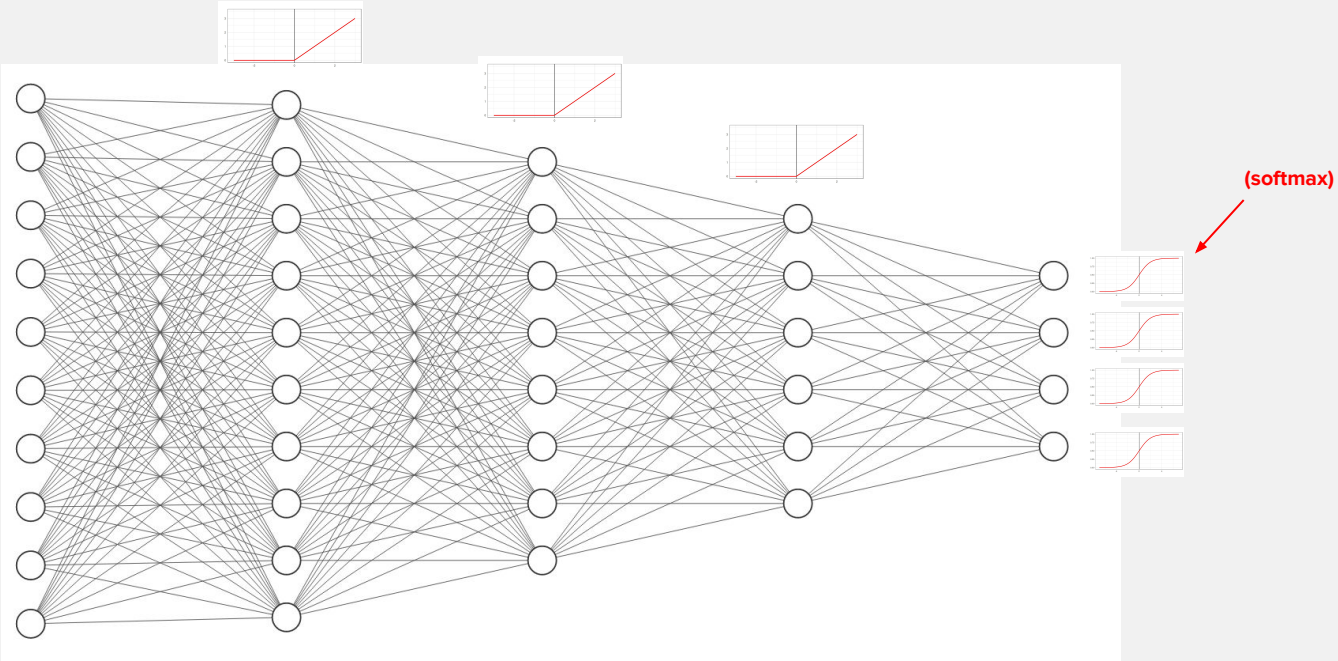
 © 2025 General Assembly

Example: Classification with NNs



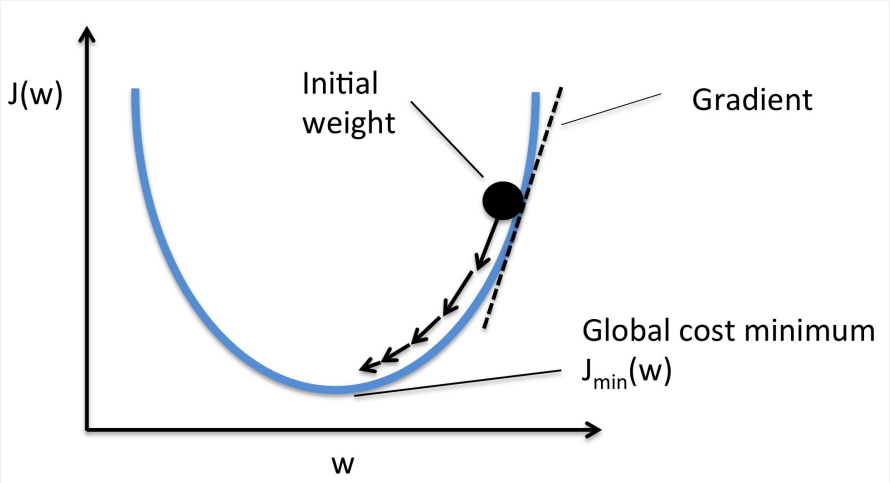
 © 2025 General Assembly

Example: Classification with NNs



Loss Functions

Neural networks are fit using **gradient descent** with respect to some **loss function** that measures how wrong your results are. These don't vary very much, and they probably look a little familiar:



 © 2025 General Assembly

Source

Loss Functions

For regression, we often pick:

$$MSE = \frac{1}{n}\|\mathbf{y} - \hat{\mathbf{y}}\|^2$$

For binary classification, we often pick **binary crossentropy**:

$$-\frac{1}{n}\sum\left(y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)\right)$$

For multiclass classification, we have **crossentropy**, which is a generalization of binary crossentropy.

 © 2025 General Assembly

Back Propagation

The reason neural networks are so computationally intensive is because of the algorithm that is used to fit them: **back propagation**, or **backprop** for short.

It involves iterating between two steps.

1

Forward pass

Compute predictions and loss.

2

Backward pass

Compute gradients based on predictions and update weights.

 © 2025 General Assembly

Forward Pass

$$x$$

$$h^{[1]} = a(\mathbf{W}^{[1]}x + b^{[1]})$$

$$h^{[2]} = a(\mathbf{W}^{[2]}h^{[1]} + b^{[2]})$$

$$\ddots$$

$$\hat{y} = a(\mathbf{W}^{[L]}h^{[L-1]} + b^{[L]})$$

 © 2025 General Assembly

Backward Pass

Compute loss

$$J(\theta) = L(y, \hat{y})$$

Update weights (let ***θ*** denote ***all*** model parameters)

$$\theta \longleftarrow \theta - \alpha \nabla J(\theta)$$

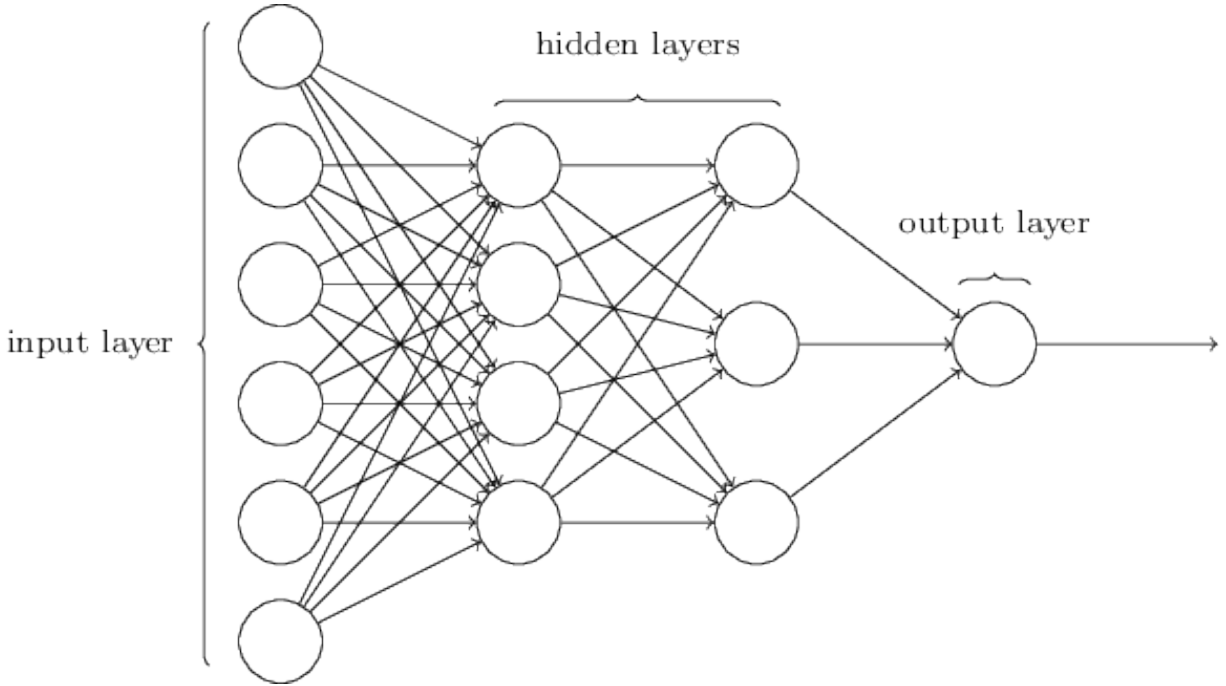
Learning rate

Vector derivative

 © 2025 General Assembly

How many parameters need to be learned?

Exclude bias/intercept term.



 © 2025 General Assembly

Why so slow...?????

Neural networks are notoriously slow to fit. Even medium sized networks can have **over a million weights** to train. How can we speed this up?

1. Get access to a better computer (preferably with a GPU).
2. Use minibatch techniques.
3. Use fancier optimization techniques.



Source

 © 2025 General Assembly

Bigger and Better: GPUs

GPUs are essentially matrix multiplication machines. Neural network training takes a fraction of the time when using one.

It is very unwise to have a production-level neural network not taking advantage of a GPU.

You can rent GPU time from every major cloud computing resource.



Source

 © 2025 General Assembly

Minibatch Techniques

Actually, the *biggest* barrier to efficient training is the fact that we're trying to use all of our data at once!

Instead, at each iteration of gradient descent, we'll use a **minibatch** of the data. A minibatch will be a smaller subsampling of the full data. One run through the full dataset is called an **epoch**. This is pronounced "epic," unless you are British, in which case it is "eepock."

For example, if you have 1,000 samples with a minibatch size of 100, you will run through 10 **training steps** in each epoch.

 © 2025 General Assembly

Minibatch Advantages

1

It's much faster.

2

Allows us to do "online learning" (train some of the model now, train more when we get more data later).

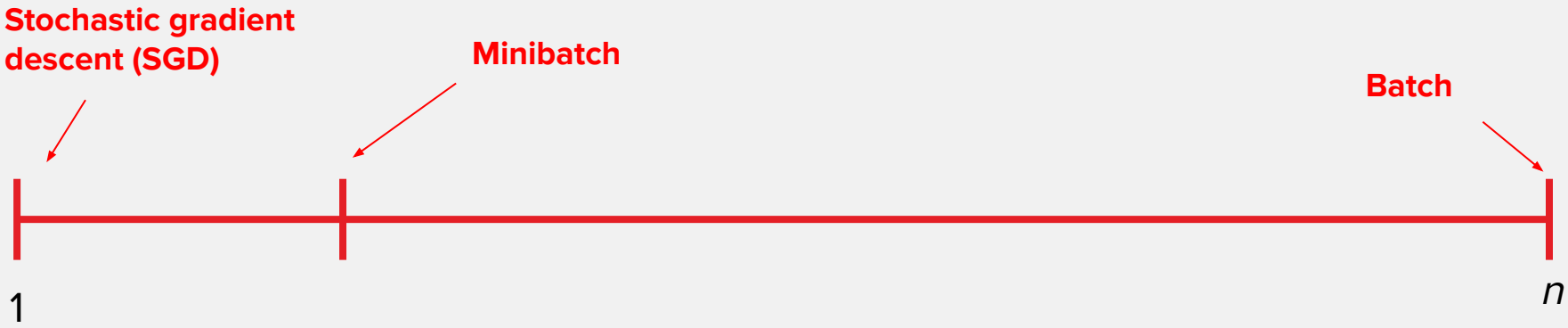
3

Helps us avoid **local minima** during optimization, which can be a big concern.

 © 2025 General Assembly

Minibatch Size

The minibatch size is typically small. It is commonly as low as 16 or 32. An interesting note: minibatch sizes are typically a power of 2, since that works best with the inner workings of GPUs.



 © 2025 General Assembly

Fancy Optimization

A lot of academic research goes into faster and more efficient techniques for gradient descent. You don't need to know how they work, but you'll want to recognize their names:

- **AdaGrad** (Adaptive Gradient)
- **RMSProp** (Root Mean Square Propagation)
- **Adam** (Adaptive Moment Estimation)

They each provide interesting updates to the usual gradient descent. You'll see **RMSProp** and **Adam** the most.

 © 2025 General Assembly

Overfitting

Finally, we might be concerned with **overfitting**. There are lots of techniques for **regularizing** your neural net. We have a whole class devoted to this:

- Penalty parameters at each hidden layer.
- Dropout.
- Early stopping.

 © 2025 General Assembly

Running notebooks in Google Colab

What is colab?

- A free, cloud-based jupyter notebook environment provided by Google. No setup required.

Why colab?

- Run python code in your browser
- Free access to GPUs
- Sharable and collaborative
- Pre-installed common libraries



How to use it for this course:

1. Go to the course GitHub repo
2. Click on any .ipynb notebook file
3. Click the **“Open in Colab”** button (or copy the URL and paste it into colab.research.google.com)
4. If needed, **save a copy to your Google Drive** to make edit

Important:

You’ll need to upload the notebooks **yourself** — changes in Colab aren’t saved back to GitHub.

See Google’s Colab site for more details, see the FAQs, or open Colab to start using it.

 © 2025 General Assembly

How can you use Jupyter Notebook in Google Colab?

Some important points about writing code in **Colab***:

1. Save your progress and work (**File -> Save**)
2. You need to be connected to the internet to use **Colab** (but you don't need to if you have Python or Jupyter installed on your machine)
3. One cell must **finish** running before another cell can run
4. A code cell won't be executed until you run the cell
5. If you close the Browser tab for your notebook, the notebook will still run in the background. To stop it running, go to the Runtime menu (**Runtime -> Disconnect and delete runtime**)

 © 2025 General Assembly

** This is different from when you use Jupyter Notebooks on your local machine*

Introduction to NNs

Let's see it for ourselves! We've cover 2 (3 if we have time) notebooks in the rest of this session

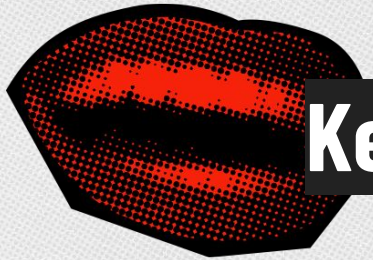
- Open them in this order: **02-oop-review.ipynb, 03-intro-to-pytorch.ipynb, 04-activation-functions.ipynb**



- What are the basic methods and attributes required for a pytorch neural network?

 © 2025 General Assembly

Bring It Home



Key takeaways

1. Neural networks have evolved from simple ideas to powering today's top AI models.
2. They learn patterns by adjusting weights through layers using data.
3. You can build them with tools like PyTorch or TensorFlow.
4. Deep learning is behind tech like voice assistants, self-driving cars, and medical AI.

 © 2025 General Assembly


GA
GENERAL ASSEMBLY